

เอกสารสรุปกระบวนการทำงาน

การพัฒนาเกม Aetheris
(โปรเจกต์ SERAAC)

Prototype build v0.1.3

จัดทำโดย

กลุ่ม MTG (MorThodGame)

1. เธียรภาส ธนะสีลังกูร รหัสนักศึกษา 682110122
2. จิรายุ โพธิ์ทอง รหัสนักศึกษา 682110107
3. อรอักษรา อักษรณรงค์ รหัสนักศึกษา 682110153
4. พงศภัค ท่าดีสม รหัสนักศึกษา 682110130

ภาคเรียนที่ 1 ปีการศึกษา 2569

มหาวิทยาลัยเชียงใหม่

คำนำ

เอกสารฉบับนี้สรุปกระบวนการทำงานของกลุ่ม MTG ในการพัฒนาเกม Aetheris (ชื่อโปรเจกต์ SERAAC) ตั้งแต่การวางแผน การออกแบบ การจัดการและแบ่งปันไฟล์ระหว่างสมาชิก ไปจนถึงการใช้ระบบควบคุมเวอร์ชัน จนได้ผลงานเวอร์ชัน Prototype build v0.1.3 ซึ่งเล่นได้ตั้งแต่บทนำจนถึงการกำจัดบอส Wyvern

เนื้อหาแบ่งเป็น 5 บท ตามหัวข้อที่กำหนด ได้แก่ แผนการดำเนินงาน รายละเอียดการออกแบบ การจัดการและแบ่งปันไฟล์ การพัฒนาบนเครื่องมือ version control และบทสรุป ภาพประกอบทั้งหมดมาจากเครื่องมือและไฟล์งานจริงของโปรเจกต์

คณะผู้จัดทำ
กลุ่ม MTG

สารบัญ

หน้า

คำนำ 2

สารบัญ 3

สารบัญภาพ 5

สารบัญตาราง 6

บทที่ 1 แผนการดำเนินงาน 7

1.1 วิธีวางแผนและเครื่องมือที่ใช้ 7

1.2 ไทม์ไลน์การทำงานจริง 7

1.3 กระดานงาน (Kanban board) 8

1.4 การแบ่งหน้าที่ 9

บทที่ 2 รายละเอียดการออกแบบ 10

2.1 เอกสารการออกแบบ 10

2.2 ตัวละคร 10

2.3 ฉาก พื้นหลัง และเมนู 12

2.4 แผนที่ 13

2.5 ผลลัพธ์ในเกม 14

2.6 การปรับการออกแบบจากผล Playtest ครั้งที่ 1 15

บทที่ 3 การจัดการและแบ่งปันไฟล์ 16

3.1 ช่องทางที่ใช้ 16

3.2 การใช้ Discord 16

3.3 โครงสร้างไฟล์ในโปรเจกต์ 17

3.4 สรุปผลการใช้ช่องทางสื่อสารและแบ่งปันไฟล์ 17

บทที่ 4 การพัฒนาบนเครื่องมือ version control 19

4.1 การใช้ Git และ GitHub 19

4.2 ประวัติการ commit 19

4.3 ข้อสังเกตเกี่ยวกับการใช้ version control 20

4.4 Release เวอร์ชันส่งงาน 21

บทที่ 5 สรุป **22**

สารบัญภาพ

หน้า

ภาพที่ 1.1 ไทม์ไลน์งานหลักของโปรเจกต์ (Gantt chart) แยกสีตามประเภทงาน 8
ภาพที่ 1.2 กระดานงานแบบ Kanban ณ เวอร์ชัน Prototype build v0.1.3 9
ภาพที่ 2.1 ภาพพอร์ตเทรตของ Relle และ Naomi (Photoshop) 11
ภาพที่ 2.2 สไปรต์ต่อสู้ (ท่ายืน) และสไปรต์เดินสำรวจของ Relle และ Naomi 11
ภาพที่ 2.3 ศัตรูในเกม: Gnawer, Husk, Wraith และบอส Wyvern 11
ภาพที่ 2.4 พื้นหลังฉากต่อสู้ City Outskirts (ซ้าย) และลานต่อสู้บอส Wyvern ใน Fallen Academy (ขวา) 12
ภาพที่ 2.5 ลานจอดและเครื่อง Osprey ที่ render จาก Blender ใช้ในฉาก Airlift Platform 12
ภาพที่ 2.6 ภาพหน้าเมนูหลัก ประกอบจากเลเยอร์ที่ทำใน Photoshop 12
ภาพที่ 2.7 แผนที่ห้องของ Relle (ซ้าย) และ Airlift Platform (ขวา) ที่ออกแบบใน Tiled 13
ภาพที่ 2.8 บทนำ: บทสนทนา ห้องของ Relle การซ้อมในเครื่องจำลอง และ Airlift Platform 14
ภาพที่ 2.9 The Sanctum หน้าจอโหลด การสำรวจ Fallen Academy และการต่อสู้กับ Wyvern 14
ภาพที่ 3.1 ช่องต่าง ๆ ในเซิร์ฟเวอร์ Discord ของทีม 16
ภาพที่ 3.2 ตัวอย่างการแชร์ไฟล์และให้ความเห็นงานใน Discord 17
ภาพที่ 3.3 โครงสร้างโฟลเดอร์หลักของ repository 17
ภาพที่ 4.1 โครงสร้าง branch ของ repository 19
ภาพที่ 4.2 จำนวน commit รายสัปดาห์ แยกตามผู้ commit (วันที่คือวันจันทร์ของสัปดาห์) 20
ภาพที่ 4.3 หน้า Release v0.1.3 บน GitHub 21

สารบัญตาราง

หน้า

ตารางที่ 1.1 กำหนดการ Sprint ตามแผน (จาก docs/agile/02-sprint-backlog.md) 7
ตารางที่ 1.2 บทบาทในทีมตามชื่อ branch 9
ตารางที่ 2.1 เอกสารการออกแบบของโปรเจกต์ 10
ตารางที่ 2.2 ความคิดเห็นจาก Playtest ครั้งที่ 1 และสิ่งที่ปรับ 15
ตารางที่ 3.1 ช่องทางการสื่อสารและแบ่งปันไฟล์ 16
ตารางที่ 4.1 จำนวน commit ของผู้ร่วมพัฒนา 20

บทที่ 1
แผนการดำเนินงาน

1.1 วิธีวางแผนและเครื่องมือที่ใช้

ทีมวางแผนงานแบบ Agile แบ่งงานทั้งเทอมเป็น 4 Sprint (Sprint ละ 2 สัปดาห์) เอกสารวางแผนทั้งหมดเขียนเป็นไฟล์ Markdown และเก็บไว้ใน repository บน GitHub ในโฟลเดอร์ `docs/agile` ได้แก่ Product Backlog (รายการ User Story ทั้งหมดพร้อมลำดับความสำคัญแบบ MoSCoW) Sprint Backlog (Story ที่จะทำในแต่ละ Sprint) Sprint Plan (ผู้รับผิดชอบ ค่าประมาณงาน และสถานะของแต่ละงาน) และบันทึกการประชุม Stand-up รายสัปดาห์

การติดตามงานในแต่ละวันทำผ่าน Discord โดยสมาชิกแจ้งความคืบหน้า ส่งภาพหรือไฟล์ที่ทำเสร็จ และนัดประชุมเสียงเมื่อต้องตัดสินใจร่วมกัน จากนั้นจึงนำสถานะไปอัปเดตใน Sprint Plan

ตารางที่ 1.1 กำหนดการ Sprint ตามแผน (จาก docs/agile/02-sprint-backlog.md)

Sprint	เริ่ม	สิ้นสุด	เป้าหมายหลัก
Sprint 1	1 ก.ย. 2569	14 ก.ย. 2569	Concept, GDD และต้นแบบระบบพื้นฐาน (เลเวล ศัตรู ทางเลือก)
Sprint 2	15 ก.ย. 2569	28 ก.ย. 2569	ทำ Core Mechanic ให้เล่นได้ (การต่อสู้ ของดรอป ภาพประกอบ)
Sprint 3	29 ก.ย. 2569	12 ต.ค. 2569	ต้นแบบ SERAAC ตามดีไซน์ใหม่ และ Playtest ครั้งที่ 1
Sprint 4	13 ต.ค. 2569	26 ต.ค. 2569	งานภาพ เสียง และปรับสมดุลเกม

1.2 ไทม์ไลน์การทำงานจริง

ภาพที่ 1.1 แสดงไทม์ไลน์ของงานหลักที่เกิดขึ้นจริง รวบรวมจาก Sprint Plan ประวัติ commit และวันที่ของไฟล์งาน งานช่วงแรกเป็นต้นแบบเดิม (ดันเจี้ยนและการต่อสู้) ต่อมาปลายเดือนกันยายนทีมตัดสินใจออกแบบเกมใหม่เป็น SERAAC จึงมีการทำต้นแบบใหม่และทดสอบกับผู้เล่นในต้นเดือนตุลาคม หลังจากนั้นงานภาพ เสียง และบทนำของเกมขยายต่อเนื่องเลย Sprint 4 ไปจนถึงต้นเดือนพฤศจิกายน เพื่อให้ได้เวอร์ชันที่พร้อมสำหรับ Playtest ครั้งที่ 2



ภาพที่ 1.1 ไทม์ไลน์งานหลักของโปรเจกต์ (Gantt chart) แยกสีตามประเภทงาน

1.3 กระดานงาน (Kanban board)

ภาพที่ 1.2 สรุปสถานะงาน ณ เวอร์ชัน v0.1.3 ในรูปแบบ Kanban board แบ่งเป็นงานที่ต้องทำ งานที่กำลังทำ และงานที่เสร็จแล้ว รายการงานรวบรวมจาก Sprint Plan และการคุยงานใน Discord



ภาพที่ 1.2 กระดานงานแบบ Kanban ณ เวอร์ชัน Prototype build v0.1.3

1.4 การแบ่งหน้าที่

สมาชิกแต่ละคนรับผิดชอบงานตามบทบาท ซึ่งสะท้อนในชื่อ branch บน GitHub ที่ใช้รูปแบบ `ชื่อเล่น/งาน/บทบาท`

ตารางที่ 1.2 บทบาทในทีมตามชื่อ branch

Branch	งาน	บทบาท
Peach/Game1/programmer, Peach/Level-Progression-System/programmer	ระบบหลักของเกม ระบบเลเวล	Programmer
jo/level-progression-system/designer	ออกแบบระบบเลเวลและเอกสาร GDD	Designer
Fishe/game-visual/Artist, Tap/game_visual/Artist	ภาพในเกม สไปรต์ และแอนิเมชัน	Artist

บทที่ 2
รายละเอียดการออกแบบ

2.1 เอกสารการออกแบบ

เอกสารการออกแบบหลักเก็บไว้ใน repository เพื่อให้ทุกคนเปิดดูฉบับล่าสุดได้ตรงกัน ส่วนการออกแบบงานภาพทำใน Adobe Photoshop และ Blender แล้วส่งภาพตัวอย่างให้ทีมดูใน Discord ก่อนนำเข้าเกม

ตารางที่ 2.1 เอกสารการออกแบบของโปรเจกต์

เอกสาร	ที่เก็บ	เนื้อหา
Game Design Document	`docs/01_GDD/`	Concept, core loop, ขอบเขตฟีเจอร์, กลไกเกม, class diagram, รายการ asset และ pipeline
Agile documents	`docs/agile/`	Product backlog, sprint backlog, sprint plan, บันทึกการประชุม
PROTOTYPE.md	`SERAAC/`	คำอธิบายระบบในเกมเทียบกับเอกสารออกแบบ และวิธีปรับค่า
Balance sheet	`SERAAC/Docs/`	ค่าสถานะตัวละคร ศัตรู สูตรดาเมจ ผลจำลองการเล่น (ไทย/อังกฤษ)
แบบฟอร์ม Playtest	`SERAAC/Docs/`	แบบทดสอบบทนำจนถึงการกำจัด Wyvern และตารางสรุปผล
ไฟล์ต้นฉบับงานภาพ	เครื่องของผู้ออกแบบ	ไฟล์ .psd (Photoshop), .blend (Blender), .tmx (Tiled)

2.2 ตัวละคร

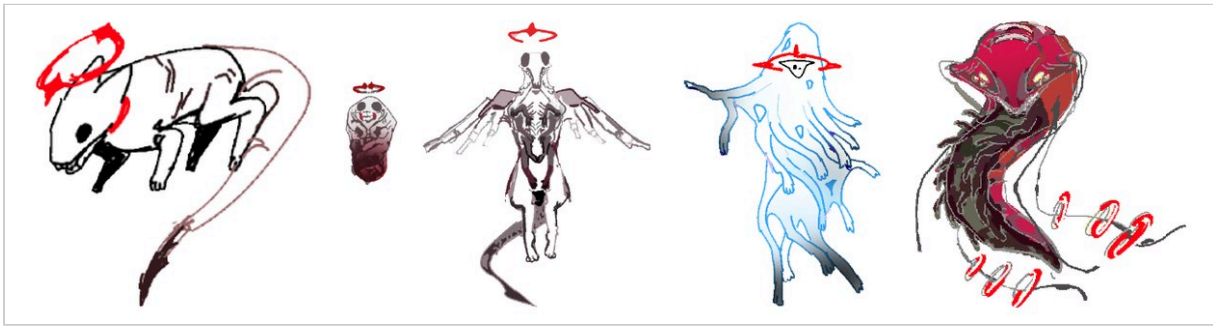
ตัวละครหลักออกแบบใน Photoshop เป็นสองแบบ แบบแรกคือภาพพอร์ตเทรตความละเอียดสูงสำหรับฉากบทสนทนาและกรอบหน้าตัวละครในหน้าต่อสู้ แบบที่สองคือสไปรต์พิกเซลสำหรับฉากต่อสู้ (แอนิเมชันท่ายืน 6 เฟรม) และสไปรต์ขนาดเล็กสำหรับการเดินสำรวจ



ภาพที่ 2.1 ภาพพอร์ตเทรตของ Relle และ Naomi (Photoshop)



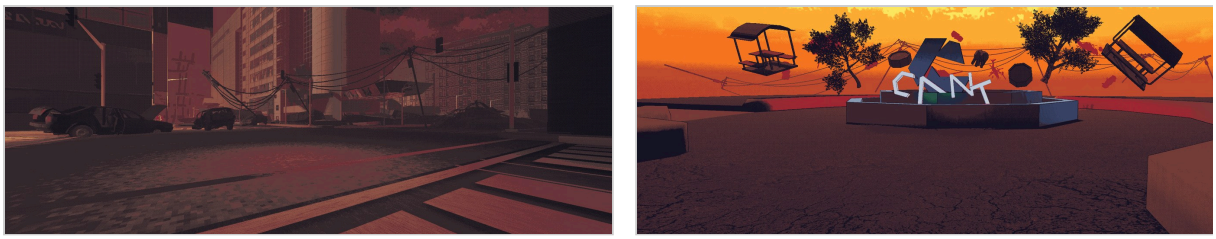
ภาพที่ 2.2 สไปรต์ต่อสู้ (ท่ายืน) และสไปรต์เดินสำรวจของ Relle และ Naomi



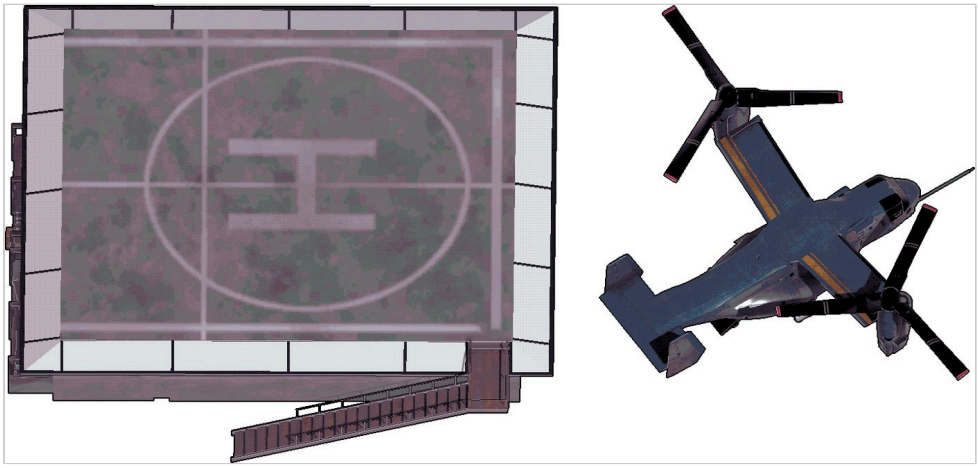
ภาพที่ 2.3 ศัตรูในเกม: Gnawer, Husk, Wraith และบอส Wyvern

2.3 ฉาก พื้นหลัง และเมนู

พื้นหลังฉากต่อสู้แบ่งเป็นหลายชั้น (backdrop, background, middleground) เพื่อให้กล้องในเกมเลื่อนแบบ parallax ได้ ฉากของบอสและลานจอดเฮลิคอปเตอร์สร้างเป็นโมเดล 3D ใน Blender แล้ว render ออกมาเป็นภาพสำหรับใช้ในเกม



ภาพที่ 2.4 พื้นหลังฉากต่อสู้ City Outskirts (ซ้าย) และลานต่อสู้บอส Wyvern ใน Fallen Academy (ขวา)



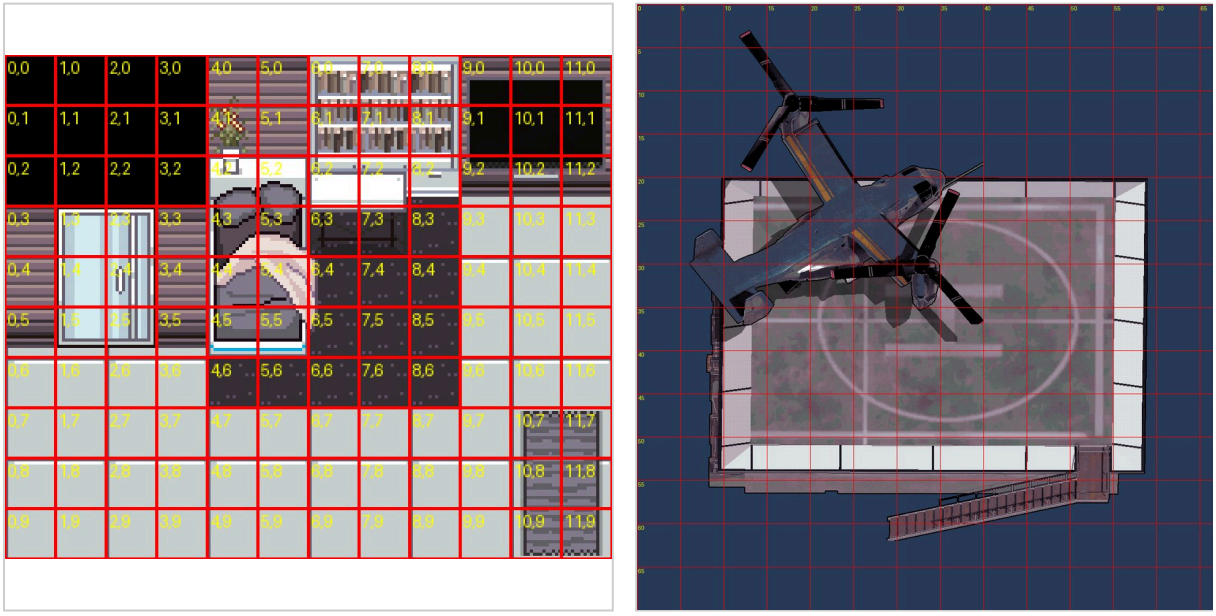
ภาพที่ 2.5 ลานจอดและเครื่อง Osprey ที่ render จาก Blender ใช้ในฉาก Airlift Platform



ภาพที่ 2.6 ภาพหน้าเมนูหลัก ประกอบจากเลเยอร์ที่ทำใน Photoshop

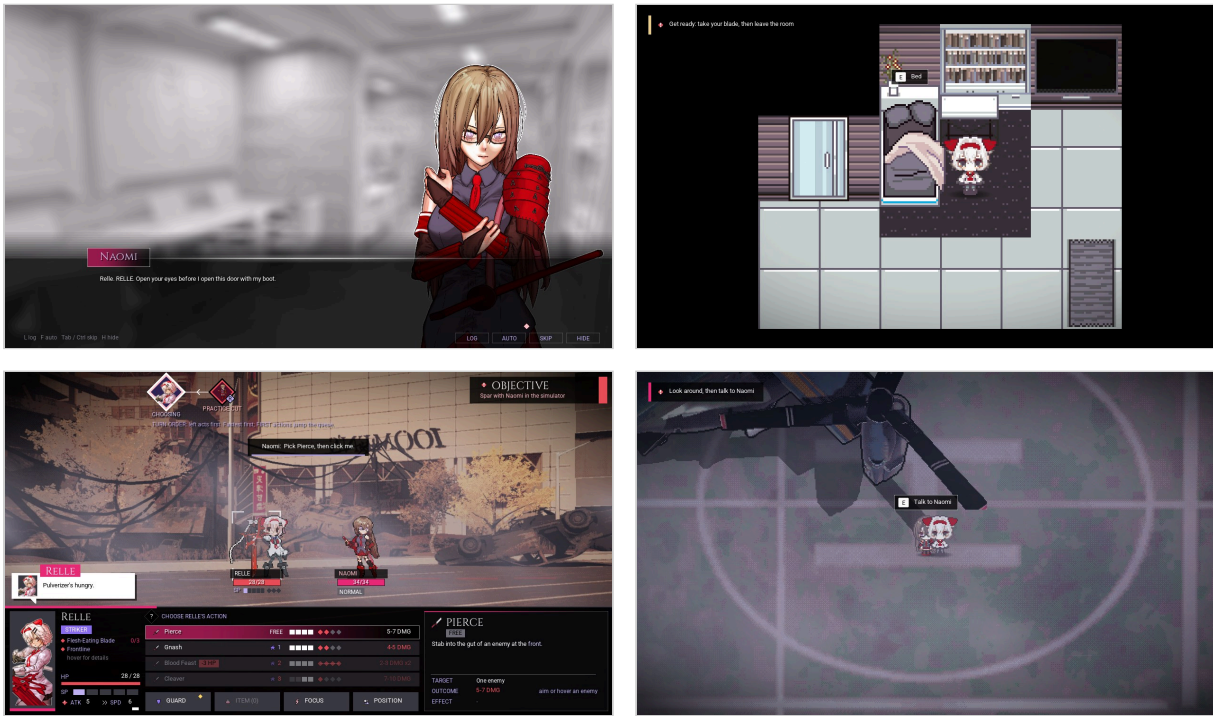
2.4 แผนที่

แผนที่ที่ให้ผู้เล่นเดินสำรวจออกแบบด้วยโปรแกรม Tiled โดยวาด tile ลงบนเลเยอร์ แล้วเพิ่ม object layer สำหรับขอบเขตการชน พื้นที่เดินได้ จุดโต้ตอบ ตำแหน่ง NPC และจุดเกิดของผู้เล่น เกมอ่านไฟล์ .tmx โดยตรงตอนรัน ทำให้แก้แผนที่แล้วเห็นผลในเกมทันที

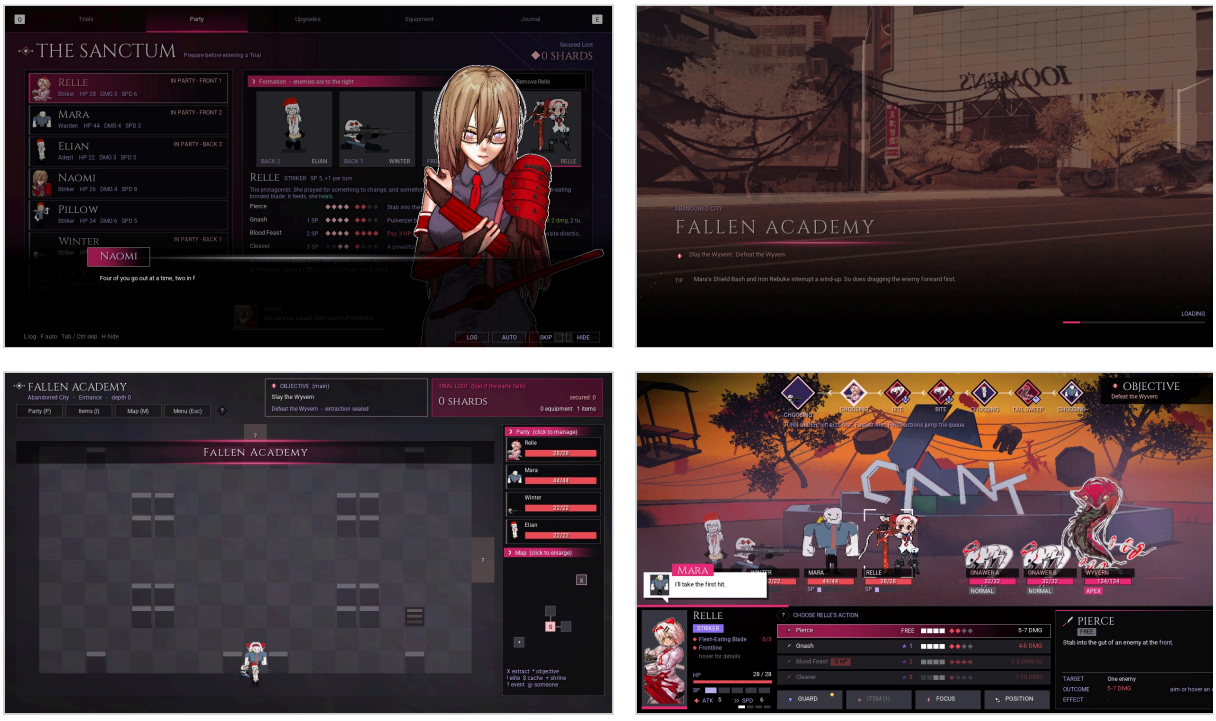


ภาพที่ 2.7 แผนที่ห้องของ Relle (ซ้าย) และ Airlift Platform (ขวา) ที่ออกแบบใน Tiled

2.5 ผลลัพธ์ในเกม



ภาพที่ 2.8 บทนำ: บทสนทนา ห้องของ Relle การซ้อมในเครื่องจำลอง และ Airlift Platform



ภาพที่ 2.9 The Sanctum หน้าจอโหลด การสำรวจ Fallen Academy และการต่อสู้กับ Wyvern

2.6 การปรับการออกแบบจากผล Playtest ครั้งที่ 1

Playtest ครั้งที่ 1 ใช้การสัมภาษณ์ผู้ทดสอบ 5 คนหลังเล่น ผลที่พบซ้ำมากที่สุดคือ UI และปุ่มควบคุมเข้าใจยาก ทีมจึงนำผลมาปรับการออกแบบตามตารางที่ 2.2

ตารางที่ 2.2 ความคิดเห็นจาก Playtest ครั้งที่ 1 และสิ่งที่ปรับ

ความคิดเห็น	สิ่งที่ปรับในเวอร์ชันนี้
UI ดูยาก ปุ่มแปลก เข้าใจยาก	ออกแบบ UI การต่อสู้ใหม่ มีไอคอน คำอธิบายเมื่อชี้เมาส์ และบทนำที่สอนทีละขั้น
อยากให้มีปุ่มเร่งความเร็ว (x2)	กดค้างเพื่อเร่งการต่อสู้ และข้ามบทสนทนาได้
ช่วงสำรวจไม่มีอะไรทำ	ศัตรูเดินในห้องและไล่ตาม มีของให้เก็บ จุดพัก และเหตุการณ์
อยากให้กราฟิกชัดขึ้น และมีสีแดงมากขึ้น	พื้นหลังแบบ parallax หมอกและฝุ่น พาเลตสีโทนแดงทั้งเกม
อยากให้ศัตรูหลากหลายขึ้น	ศัตรู 3 ประเภทที่เล่นต่างกัน และบอส Wyvern

บทที่ 3
การจัดการและแบ่งปันไฟล์

3.1 ช่องทางที่ใช้

ตารางที่ 3.1 ช่องทางการสื่อสารและแบ่งปันไฟล์

ช่องทาง	ใช้ทำอะไร
Discord	ช่องทางหลักของทีม: คุยงานประจำวัน ส่งภาพตัวอย่างและไฟล์งาน (PNG, PSD, เสียง) แจ้งปัญหา และประชุมเสียง
GitHub	เก็บโค้ด เอกสารออกแบบ และเอกสาร Agile พร้อมประวัติการแก้ไข
Adobe Photoshop, Blender, Tiled	สร้างงานภาพ โมเดล 3D และแผนที่ ไฟล์ต้นฉบับเก็บในเครื่องผู้ออกแบบ แล้ว export เป็น PNG หรือ .tmx เข้าโปรเจกต์

3.2 การใช้ Discord

ทีมใช้เซิร์ฟเวอร์ Discord แยกช่องตามประเภทงาน เมื่อทำงานภาพหรือเสียงเสร็จ ผู้ทำจะส่งภาพตัวอย่างหรือไฟล์ลงในช่องที่เกี่ยวข้อง ให้สมาชิกคนอื่นให้ความเห็นได้ทันที เมื่องานผ่านแล้วจึงนำไฟล์ไปใส่ในโฟลเดอร์ `Art/` หรือ `Audio/` ของโปรเจกต์

แทรกภาพหน้าจอเซิร์ฟเวอร์ Discord ของทีม: รายชื่อช่อง (เช่น ช่องประกาศ ช่องงานภาพ ช่องโค้ด ช่องแชร์ไฟล์)

ภาพที่ 3.1 ช่องต่าง ๆ ในเซิร์ฟเวอร์ Discord ของทีม

แทรกภาพหน้าจอการส่งไฟล์ใน Discord: ไฟล์ภาพ/PSD/เสียงที่แชร์ และการคุยแก้งาน

ภาพที่ 3.2 ตัวอย่างการแชร์ไฟล์และให้ความเห็นงานใน Discord

3.3 โครงสร้างไฟล์ในโปรเจกต์

ไฟล์ทั้งหมดของเกมจัดเป็นโฟลเดอร์ตามประเภท ภาพและเสียงอยู่ในโฟลเดอร์ `Art/` และ `Audio/` ซึ่งเกมโหลดตอนรัน ผู้ออกแบบจึงเปลี่ยนภาพหรือเสียงได้โดยแค่วางไฟล์ใหม่ทับไฟล์เดิม ไม่ต้องแก้โค้ด ไฟล์ตั้งค่าที่ใช้บ่อย เช่น พื้นหลังฉากต่อสู้และเอฟเฟกต์หมอก เป็นไฟล์ JSON ที่แก้แล้วเห็นผลขณะเกมยังเปิดอยู่

```
morthodkai-inc-aac-rip-off-/                          รากของ repository
├── README.md                                         วิธี build / run / เล่น
├── [MorThodKaiInc] - game/docs/                      GDD, backlog, sprint plan, meeting notes
└── [MorThodKaiInc] - game/game/Code/Game/SERAAC/     โปรเจกต์เกม (C#)
    ├── Art/                                          ภาพ แผนที่ Tiled สไปรต์ พื้นหลัง (โหลดตอนรัน)
    ├── Audio/                                        เพลงและเสียงประกอบ
    ├── Content/                                      ฟอนต์ (MonoGame Content Pipeline)
    ├── Combat/ Core/ Data/ Screens/ Story/ UI/ World/  ซอร์สโค้ด
    └── Docs/                                         รายงาน แบบทดสอบ Balance sheet
```

ภาพที่ 3.3 โครงสร้างโฟลเดอร์หลักของ repository

3.4 สรุปผลการใช้ช่องทางสื่อสารและแบ่งปันไฟล์

การใช้ Discord เป็นช่องทางหลักได้ผลดีกับทีม เพราะสมาชิกทุกคนใช้อยู่แล้ว ส่งภาพและไฟล์ได้เร็ว ให้ความเห็นงานภาพได้ทันที และประชุมเสียงได้โดยไม่ต้องนัดสถานที่ งานภาพ เสียง และบทนำของเกมในช่วงหลังจึงคืบหน้าได้เร็ว

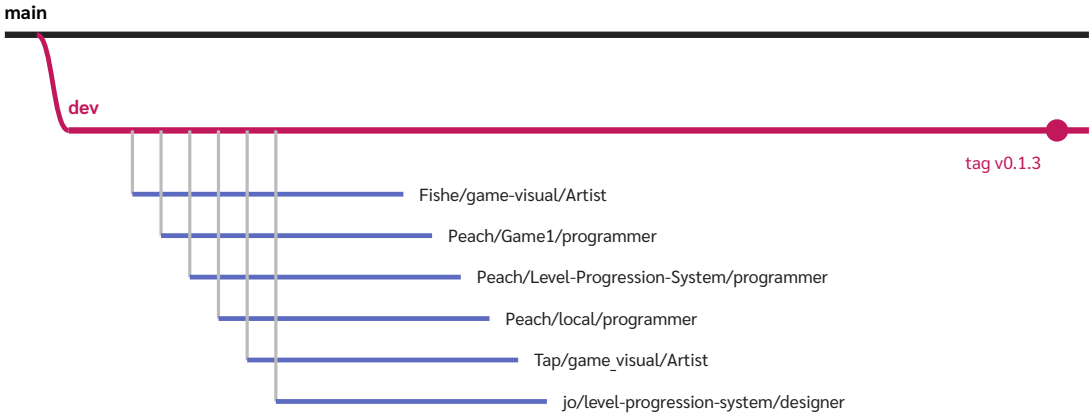
ปัญหาที่พบคือไฟล์ที่ส่งในแชตค้นหายากเมื่อเวลาผ่านไป บางครั้งไม่แน่ใจว่าไฟล์ใดเป็นฉบับล่าสุด และไฟล์ขนาดใหญ่ เช่น ไฟล์เสียงชุดใหญ่หรือไฟล์ Blender ส่งผ่าน

Discord ไม่สะดวก แนวทางที่ทีมใช้ต่อไปคือปักหมุดไฟล์ฉบับล่าสุดในแต่ละช่อง ตั้งชื่อไฟล์ตาม naming convention ในเอกสาร GDD และนำไฟล์ที่ผ่านแล้วเข้า repository ทันที

บทที่ 4
การพัฒนาบนเครื่องมือ version control

4.1 การใช้ Git และ GitHub

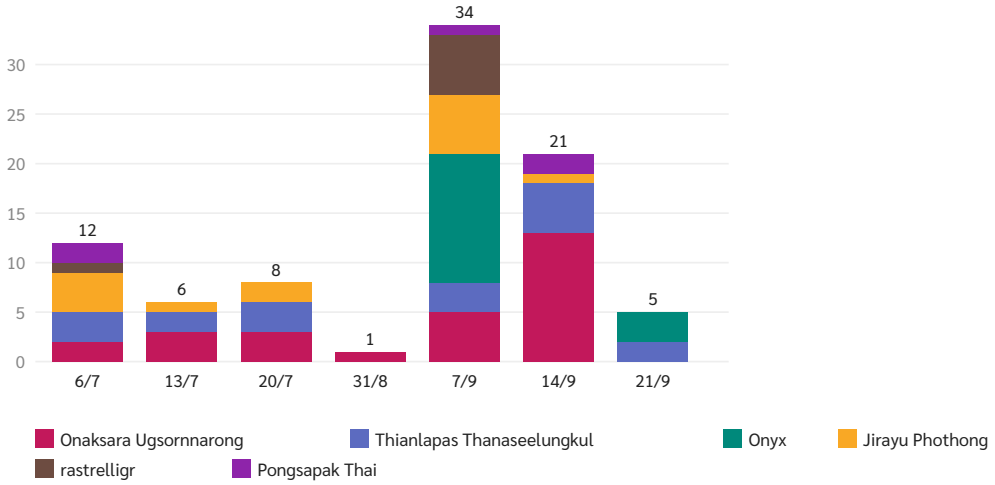
โปรเจกต์ใช้ Git ร่วมกับ GitHub (repository `rastrelligr/morthodkai-inc-aac-rip-off-`) โดยมี branch `main` สำหรับเวอร์ชันที่เสถียร และ branch `dev` สำหรับรวมงานระหว่างพัฒนา สมาชิกแต่ละคนแตก branch ของตัวเองจาก `dev` ตามรูปแบบ `ชื่อเล่น/งาน/บทบาท` แล้วรวมกลับผ่าน merge หรือ Pull Request ข้อความ commit ใช้รูปแบบ conventional commits เช่น `feat:` `fix:` `docs:` `chore:`



ภาพที่ 4.1 โครงสร้าง branch ของ repository

4.2 ประวัติการ commit

ภาพที่ 4.2 แสดงจำนวน commit รายสัปดาห์แยกตามผู้ commit และตารางที่ 4.1 แสดงจำนวน commit ทั้งหมดของแต่ละคน ข้อมูลดึงจากประวัติของ repository โดยตรง



ภาพที่ 4.2 จำนวน commit รายสัปดาห์ แยกตามผู้ commit (วันที่คือวันจันทร์ของสัปดาห์)

ตารางที่ 4.1 จำนวน commit ของผู้ร่วมพัฒนา

ผู้ commit	จำนวน commit
Onaksara Ugsornnarong	27
Thianlapas Thanaseelungkul	18
Onyx	16
Jirayu Phothong	14
rastrelligr	7
Pongsapak Thai	5

4.3 ข้อสังเกตเกี่ยวกับการใช้ version control

ช่วงเดือนกันยายนทีมใช้ GitHub อย่างสม่ำเสมอ ทั้งการแตก branch การ merge และการแก้ conflict ร่วมกัน แต่หลังจากเปลี่ยนเป็นดีไซน์ใหม่ปลายเดือนกันยายน งานส่วนใหญ่ทำบนเครื่องของผู้ทำงานหลักและแชร์ผ่าน Discord แทน ประวัติบน GitHub จึงไม่ได้สะท้อนงานในช่วงเดือนตุลาคมถึงต้นเดือนพฤศจิกายน งานทั้งหมดในช่วงนั้นถูกรวมเข้า repository อีกครั้งพร้อมกับการออกเวอร์ชัน v0.1.3

บทเรียนที่ได้คือควร commit และ push งานเป็นระยะแม้งานยังไม่เสร็จ เพื่อให้มีจุดย้อนกลับและให้สมาชิกคนอื่นเห็นความคืบหน้า ไฟล์เสียงและภาพขนาดใหญ่ควรตัดไฟล์ที่ไม่ได้ใช้ออกก่อน (ในเวอร์ชันนี้ลดโฟลเดอร์เสียงจากประมาณ 4.3 GB เหลือประมาณ 420 MB) หรือใช้ Git LFS

4.4 Release เวอร์ชันส่งงาน

เวอร์ชันส่งงานติด tag `v0.1.3` บน branch `dev` และสร้างเป็น Release บน GitHub ชื่อ Prototype build v0.1.3 เนื้อหาหลักของเวอร์ชันนี้มีดังนี้

- บทนำ 5 บท ตั้งแต่ห้องของ Relle การซ้อมต่อสู้ในเครื่องจำลอง Airlift Platform และ The Sanctum ไปจนถึงด่าน Fallen Academy และบอส Wyvern
- บทสนทนาแบบ Visual Novel พร้อมประวัติข้อความ เล่นอัตโนมัติ และข้าม
- แผนที่จาก Tiled พร้อมระบบชน จุดโต้ตอบ และ NPC
- Naomi เป็นตัวละครที่เล่นได้ และสไปรต์ใหม่ของ Relle และ Naomi
- เพลงและเสียงประกอบ หมอก ฝุ่น และเมฆ หน้าจอโหลด และหน้า Options สำหรับปรับเสียง

วิธี build รัน และเล่นเกม อธิบายไว้ในไฟล์ `README.md` ที่รากของ repository

แทรกภาพหน้าจอหน้า Releases บน GitHub ที่แสดง Release v0.1.3

ภาพที่ 4.3 หน้า Release v0.1.3 บน GitHub

บทที่ 5
สรุป

ทีมวางแผนงานด้วย Agile และเอกสารใน repository ใช้ Discord เป็นช่องทางหลักในการสื่อสารและแบ่งปันไฟล์ สร้างงานภาพด้วย Photoshop และ Blender ออกแบบแผนที่ด้วย Tiled และเก็บโค้ดกับเอกสารบน GitHub ผลลัพธ์คือ Prototype build v0.1.3 ที่เล่นได้ตั้งแต่ต้นจนจบบอสตัวแรก

สิ่งที่ทำได้ดีคือการสื่อสารผ่าน Discord ที่รวดเร็ว และการนำผล Playtest ครั้งที่ 1 มาปรับเกมอย่างเป็นรูปธรรม สิ่งที่ต้องปรับคือการใช้ version control ให้สม่ำเสมอตลอดโปรเจกต์ และการจัดเก็บไฟล์ที่แชร์ใน Discord ให้ค้นหาได้ง่าย ขั้นต่อไปของทีมคือนำผล Playtest ครั้งที่ 2 มาปรับบทนำและความยากของ Wyvern แล้วพัฒนาด่านถัดไป